

CRYPTOCURRENCY NETWORK FORENSICS USING ETHEREUM BLOCKCHAIN TRANSACTIONS

PROJECT REPORT

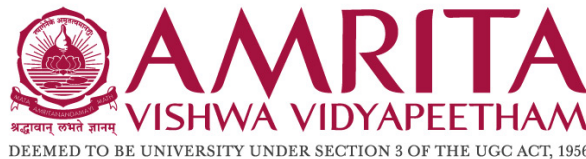
Submitted by

GROUP D05

Rangala Karthikeya	CB.SC.U4AIE23333
Tanguturi Vinay Krishna Chetty	CB.SC.U4AIE23341
Katikala Dedeepya	CB.SC.U4AIE23349
Gutta Karthikeya	CB.SC.U4AIE23358

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE



AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE – 641112 (INDIA)

MARCH – 2026

AMRITA VISHWA VIDYAPEETHAM – COIMBATORE



BONAFIDE CERTIFICATE

This is to certify that the project entitled “**Cryptocurrency Network Forensics using Ethereum Blockchain Transactions**” submitted by **Rangala Karthikeya, Tanguturi Vinay Krishna Chetty, Katikala Dedeepya and Gutta Karthikeya** is a bonafide record of the work carried out under my supervision for the award of the degree of **Bachelor of Technology in “COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE”** .

Dr. PS. Ardra

Assistant Professor

Amrita School of Artificial Intelligence

Amrita Vishwa Vidyapeetham

Coimbatore – 641112

Date: 04-03-2026

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE – 641 112

DECLARATION

We, **Rangala Karthikeya** (CB.SC.U4AIE23333), **Tanguturi Vinay Krishna Chetty** (CB.SC.U4AIE23341), **Katikala Dedeepya** (CB.SC.U4AIE23349), and **Gutta Karthikeya** (CB.SC.U4AIE23351), hereby declare that this thesis entitled “**Cryptocurrency Network Forensics using Ethereum Blockchain Transactions**” is the record of the original work carried out by us under the guidance of **Ardra PS**, Designation, Amrita School of Artificial Intelligence, Coimbatore.

To the best of our knowledge, this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or any similar award to any candidate in any University.

Place:

Signatures of the Students

Date:

Rangala Karthikeya

Tanguturi Vinay Krishna Chetty

Katikala Dedeepya

Gutta Karthikeya

COUNTERSIGNED

Acknowledgement

We would like to express our sincere gratitude to our project guide for her invaluable guidance, continuous support, and encouragement throughout this project. We are also grateful to Dr. K.P.Soman, Professor and Dean, Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, for providing us with the necessary facilities and resources. We extend our heartfelt thanks to all the faculty members of the Department of Computer Science Engineering – Artificial Intelligence for their constant motivation. Finally, we thank our family and friends for their unwavering support throughout this endeavour

Abstract

The rapid growth of blockchain technology has led to an unprecedented increase in transaction volume, making effective monitoring and analysis of blockchain networks a critical challenge. Ethereum, as one of the most widely adopted blockchain platforms, generates massive transactional data that is difficult to process using traditional analytical methods. Moreover, the open and decentralized nature of Ethereum makes it susceptible to various anomalous and fraudulent activities, including bot-driven transactions, wash trading, smurfing, zero-value transaction spam, and market manipulation schemes such as pump-and-dump events. This project presents a scalable Big Data-based framework for detecting and analyzing anomalous behavior in Ethereum blockchain transactions. The system utilizes distributed data processing using Apache Spark and PySpark to efficiently ingest, clean, and process large-scale blockchain datasets stored in Parquet format. Meaningful transaction-, wallet-, and block-level features are engineered to capture behavioral patterns across the network. A multi-dimensional analytical approach is adopted, combining rule-based detection methods, machine learning-based anomaly detection models such as Isolation Forest and Autoencoders, graph-based network analysis, and time-series analytics. This integrated methodology enables the identification of both known fraud patterns and previously unseen anomalies. Visualization techniques are employed to interpret analytical results and present insights in an intuitive manner. The proposed system successfully identifies suspicious wallets, anomalous blocks, and potential market manipulation activities, demonstrating the effectiveness of Big Data analytics and machine learning in blockchain fraud detection. This work highlights the practical applicability of scalable analytical frameworks for enhancing transparency, security, and trust in decentralized financial ecosystems.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 2.1	Spark Session Creation	15
Figure 2.2	Data Ingestion Process	16
Figure 2.3	Distributed Data Storage	16
Figure 2.4	Overall System Block Diagram	19
Figure 4.1	Spark Jobs UI	23
Figure 4.2	Spark Dataframe UI	24
Figure 4.1	Transaction Value Distribution	26
Figure 4.2	Transaction Success vs Failure	26
Figure 4.3	Gas Limit vs Gas Used	27
Figure 4.4	Whale Transaction Distribution	27
Figure 4.5	Wallets with High Failed Transactions	27
Figure 4.6	Wallet Transaction Count Distribution	28
Figure 4.7	Bot Wallet Identification	28
Figure 4.8	Repeated Sender–Receiver Pair Analysis	28
Figure 4.9	Circular Transaction Detection	29
Figure 4.10	Isolation Forest Wallet-Level Anomalies	29
Figure 4.11	Isolation Forest Block-Level Anomalies	29
Figure 4.12	Autoencoder-Based Wallet Anomalies	30
Figure 4.13	Money Laundering Pattern Detection	30
Figure 4.14	Pump-and-Dump Transaction Volume	30
Figure 4.15	Pump-and-Dump ETH Volume	31

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Schema of Blocks Dataset	12
Table 1.2	Schema of Transactions Dataset	13

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AIG	Account Interaction Graph
API	Application Programming Interface
CSV	Comma-Separated Values
DApp	Decentralized Application
DeFi	Decentralized Finance
DNN	Deep Neural Network
EDA	Exploratory Data Analysis
EIP	Ethereum Improvement Proposal
ETH	Ether (Ethereum cryptocurrency unit)
ETL	Extract, Transform, Load
FTDNet	Fraudulent Transaction Detection Network
GNN	Graph Neural Network
GPL	General Public License
GPU	Graphics Processing Unit
HDFS	Hadoop Distributed File System
I/O	Input/Output
IoT	Internet of Things
ML	Machine Learning
Parquet	Apache Parquet (columnar storage format)
PySpark	Python API for Apache Spark
ReLU	Rectified Linear Unit
SQL	Structured Query Language
URL	Uniform Resource Locator
WEI	Smallest denomination of Ether ($1 \text{ ETH} = 10^{18} \text{ Wei}$)
XGBoost	Extreme Gradient Boosting

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. Problem Statement	2
1.2. Objective	3
1.3. Literature Survey	5
1.4. Research Gap and Novelty	7
1.4.1. Research Gap	7
1.4.2. Novelty of the Project	9
1.5. Dataset Description	11
1.5.1. Source and Format	11
1.5.2. Time Coverage	12
1.5.3. Blocks Dataset	12
1.5.4. Transactions Dataset	13
1.5.5. Dataset Characteristics	14
1.5.6. Usefulness for the Project	14
1.5.7. Reference and Link	14
2. PROPOSED WORK	15
2.1. Data Collection and Ingestion	15
2.2. Data Storage and Preprocessing	16
2.3. Feature Engineering	16
2.3.1. Transaction-Level Features	17
2.3.2. Wallet-Level Features	17
2.3.3. Block-Level Features	17
2.4. Exploratory Data Analysis	17
2.5. Rule-Based Anomaly Detection	18
2.6. Machine Learning-Based Anomaly Detection	18
2.6.1. Isolation Forest	18
2.6.2. Autoencoder-Based Detection	18
2.7. Graph-Based Network Analysis	19
2.8. Time-Series and Temporal Analysis	19
2.9. Visualization and Interpretation	19
2.10. Big Data Tools and Analytics Framework	19
2.10.1. Tools and Technologies Used	20

2.11. 5V's of Big Data	22
3. DETECTIONS AND ANALYTICS PERFORMED	24
3.1. Transaction-Level Analytics	24
3.2. Wallet-Level Behavioral Analysis	24
3.3. Temporal and Time-Series Analysis	24
3.4. Unsupervised Machine Learning-Based Detection	24
3.5. Deep Learning-Based Anomaly Detection	25
3.6. Graph-Based Network Analysis	25
3.7. Detected Anomaly Categories	25
4. RESULTS AND DISCUSSION	26
4.1. Exploratory Data Analysis (EDA) Results	26
4.1.1. Transaction Value Distribution	26
4.1.2. Transaction Success vs Failure	26
4.1.3. Gas Limit vs Gas Used	27
4.2. Transaction-Level Anomalies (Whale Transactions)	27
4.3. Failed Transaction Analysis (Wallet-Level Anomaly)	27
4.4. Address-Level Behavioral Analysis	28
4.5. Bot Wallet Detection	28
4.6. Wash Trading Indicator (Repeated Sender–Receiver Pairs)	28
4.7. Wash Trading Wallet Detection (Circular Transactions)	29
4.8. Isolation Forest-Based Anomaly Detection	29
4.8.1. Wallet-Level Anomalies	29
4.8.2. Block-Level Anomalies	29
4.9. Autoencoder-Based Anomaly Detection	30
4.10. Money Laundering Patterns	30
4.11. Pump-and-Dump Detection	30
4.12. Future Work	31
4.13. Conclusion	32
REFERENCES	33
APPENDIX	34

Chapter 1

Introduction

Blockchain technology has revolutionized the way decentralized financial systems operate by enabling transparent, immutable, and trustless transactions. Among various blockchain platforms, Ethereum has emerged as one of the most widely used ecosystems, supporting smart contracts, decentralized applications (DApps), decentralized finance (DeFi), and non-fungible tokens (NFTs). As Ethereum processes millions of transactions daily, it generates massive volumes of data that require scalable and efficient analysis techniques. Despite its transparency, the Ethereum blockchain is vulnerable to various forms of malicious activities and anomalous behaviors, such as bot-driven transactions, wash trading, smurfing, zero-value transaction spam, and market manipulation strategies like pump-and-dump schemes. These activities can distort transaction statistics, mislead investors, increase network congestion, and undermine trust in decentralized systems. Traditional data processing and fraud detection methods are inadequate for handling the scale, velocity, and complexity of blockchain data. This project focuses on building a Big Data-driven anomaly detection and analytics system for Ethereum blockchain transactions. The system leverages Apache Spark and distributed data processing techniques to efficiently handle large-scale blockchain datasets stored in Parquet format. By using PySpark, the project performs high-performance data ingestion, cleaning, transformation, and aggregation across millions of transaction records. To effectively identify suspicious behaviors, the project adopts a multi-layered analytical approach. Rule-based detection techniques are used to capture known fraud patterns such as bot activity, smurfing wallets, and zero-value transaction flooding. In addition, machine learning-based anomaly detection models, including Isolation Forest and Autoencoders, are employed to detect previously unknown or subtle anomalies that cannot be identified using fixed thresholds alone. Furthermore, the project incorporates graph-based network analysis, where Ethereum wallets are modeled as nodes and transactions as edges. This enables the detection of complex behaviors such as circular fund transfers, wash trading, and coordinated wallet interactions. Time-series analytics is also applied to transaction volume and value trends to identify pump-and-dump patterns, which are indicative of market manipulation. The results of the analysis

are presented through comprehensive visualizations and analytical dashboards, allowing clear interpretation of detected anomalies and fraud patterns. The final system provides actionable insights by classifying wallets and blocks into normal and suspicious categories, thereby supporting blockchain monitoring, fraud investigation, and regulatory analysis. Overall, this project demonstrates how Big Data analytics, machine learning, and network analysis can be integrated to build a scalable, intelligent, and real-world blockchain anomaly detection system capable of handling the challenges posed by modern decentralized financial ecosystems.

1.1 Problem Statement

The rapid growth of the Ethereum blockchain has led to the generation of massive volumes of heterogeneous and high-velocity transaction data, making manual monitoring and traditional fraud detection techniques ineffective. Existing approaches are often limited to single-level analysis (transaction-level or account-level), rule-based detection mechanisms, or small-scale datasets. Such methods fail to capture complex and evolving behaviors such as whale transactions, automated bot activity, wash trading, money laundering patterns, and temporal market manipulation schemes.

Therefore, there is a critical need for a scalable, big-data-driven analytical framework that can efficiently process large-scale Ethereum transaction data and perform multi-level anomaly detection. By leveraging distributed computing and machine learning techniques, the proposed system aims to accurately identify suspicious activities in real time or near real time, thereby enhancing blockchain security, transparency, and trust.

1.2 Objective

The primary objective of this project is to design and implement a scalable Big Data analytics system for detecting anomalous and fraudulent activities in Ethereum blockchain transactions. The specific objectives of the project are as follows:

- **To collect and manage large-scale Ethereum blockchain data**
 - To obtain Ethereum block and transaction data available in Parquet format.
 - To handle the high volume, velocity, and variety of blockchain data.
 - To organize and store the data efficiently for large-scale analysis.
- **To perform distributed data ingestion and processing**
 - To load blockchain data into distributed DataFrames.
 - To process millions of transaction records efficiently.

- To leverage Apache Spark and PySpark for parallel data processing.
- **To engineer meaningful analytical features**
 - To extract transaction-level features such as transaction value, gas usage, and success status.
 - To derive wallet-level behavioral features such as transaction frequency, total inflow and outflow, and failure rate.
 - To compute block-level metrics including block size and transaction density.
 - To transform raw blockchain data into structured features suitable for analytics and modeling.
- **To detect known fraudulent patterns using rule-based techniques**
 - To identify bot-like wallets based on unusually high transaction frequency.
 - To detect smurfing behavior involving splitting transactions across multiple wallets.
 - To identify wash trading patterns through repetitive circular transfers.
 - To detect zero-value transaction spam commonly used for network manipulation.
- **To identify unknown anomalies using machine learning models**
 - To apply unsupervised machine learning techniques such as Isolation Forest and Autoencoders.
 - To detect abnormal wallet and transaction behavior without labeled data.
 - To complement rule-based detection with intelligent anomaly detection methods.
- **To perform graph-based transaction network analysis**
 - To represent Ethereum transactions as a graph of wallets and fund transfers.
 - To analyze wallet interaction patterns to identify circular flows and coordinated behavior.
 - To uncover complex fraud patterns that are not visible through individual transactions.
- **To analyze temporal behavior and market manipulation**
 - To perform time-series analysis on transaction volumes and values.
 - To detect sudden spikes or drops indicating pump-and-dump activity.
 - To study how anomalous behavior evolves over time on the blockchain.

1.3 Literature Survey

Ethereum blockchain networks generate massive volumes of transaction data, which cybercriminals increasingly exploit to carry out fraudulent activities and anomalous schemes. Several research studies have explored Ethereum analysis using graph-based approaches, machine learning methods, and deep learning techniques. This section presents five significant research studies related to the proposed work and highlights the research gap addressed by this project.

A. Behavior-aware Account De-anonymization on Ethereum Interaction Graph (2022)

Wu et al.[1] proposed a behavior-aware system for Ethereum account de-anonymization by analyzing user interaction patterns. The authors constructed an Ethereum Account Interaction Graph (AIG), where accounts are represented as nodes and transaction relationships as edges. A hierarchical graph attention encoder was employed to capture behavioral patterns across multiple layers. Contrastive self-supervised learning was used to overcome the challenge of limited labeled data.

Although the system effectively identifies account identities based on interaction behavior, it primarily focuses on de-anonymization rather than detecting fraudulent or anomalous activities. In contrast, the proposed project utilizes graph-based concepts as a supporting component to identify malicious behaviors such as bot activity, wash trading, smurfing, and pump-and-dump schemes by combining graph analysis with Big Data processing, machine learning, and time-series analytics.

B. Blockchain-based Fraud Detection: A Systematic Review of Ethereum Network Applications (2025)

Patel et al.[2] conducted a systematic review of fraud detection techniques applied to the Ethereum network. The study examined a wide range of machine learning and deep learning models, including Random Forest, Support Vector Machines, XGBoost, and neural networks. Various datasets, feature engineering strategies, evaluation metrics, and reported accuracies were compared across multiple studies.

While the review provides a comprehensive overview of existing approaches, it does not propose or implement a detection framework. Additionally, most reviewed studies rely on small, labeled datasets. The proposed project addresses these limitations by implementing a scalable, end-to-end analytics pipeline using real Ethereum blockchain data, distributed processing, and unsupervised anomaly detection techniques.

C. Aparecium: Detecting Scam Behaviors on Ethereum via Biased Random Walk (2023)

Zhang et al.[3] introduced Aparecium, a graph-based scam detection framework for Ethereum. The system models transactions as a graph and applies biased random walk sampling to preserve scam-specific structural patterns. Graph-derived features were then used to train machine learning classifiers for scam detection.

Although the approach effectively detects scam-related addresses, it focuses only on specific scam behaviors and graph-based features. The proposed project extends beyond scam detection by identifying multiple types of anomalies, including bot activity, smurfing, wash trading, and abnormal transaction bursts, using a hybrid approach combining rule-based logic, machine learning, and time-series analysis.

D. Detection of Fraudulent Transactions in Ethereum Blockchain Smart Contracts Using Deep Learning (2025)

Kumar et al.[4] developed a deep learning-based fraud detection model for Ethereum smart contracts. Their approach integrates Jaya-based feature selection with a deep neural network, referred to as FTDNet, to classify transactions as fraudulent or legitimate. High classification accuracy was achieved on labeled benchmark datasets.

Despite strong performance, the model depends heavily on labeled data and is limited to smart contract transactions. The proposed project overcomes these constraints by applying unsupervised anomaly detection techniques such as Isolation Forest and Autoencoders to identify unknown fraud patterns across all Ethereum transactions without requiring labeled data.

E. Intelligent Fraud Detection in Ethereum Transactions Using Machine Learning (2025)

Reddy et al.[5] investigated traditional supervised machine learning models, including Random Forest, XGBoost, and Support Vector Machines, to classify Ethereum transactions as fraudulent or normal. The study focused on transaction-level features such as gas usage, transaction value, and timestamps.

While the approach achieved good classification performance, it suffers from scalability limitations and relies on supervised learning assumptions. The proposed project improves upon this by leveraging Apache Spark for large-scale data processing, incorporating wallet-level and block-level behavioral features, and applying unsupervised and graph-based analytics to detect complex fraud patterns.

1.4 Research Gap and Novelty

1.4.1 Research Gap

Despite extensive research on fraud and anomaly detection in the Ethereum ecosystem, several critical gaps remain in the existing literature:

- **Limited Scalability to Real-World Blockchain Data:** Most existing studies evaluate their models on small, curated, or labeled datasets. These approaches do not scale effectively to real Ethereum transaction volumes involving millions of records, limiting their applicability in real-world blockchain monitoring systems.
- **Dependence on Supervised Learning and Labeled Data:** A significant portion of prior work relies on supervised machine learning or deep learning models that require labeled fraud datasets. However, labeled blockchain fraud data is scarce, incomplete, and often outdated, making supervised approaches impractical for detecting newly emerging fraud patterns.
- **Single-Dimensional Detection Approaches:** Many studies focus on a single analytical dimension, such as graph analysis, transaction classification, or smart contract inspection. Such isolated approaches fail to capture the complex nature of blockchain fraud, which often spans wallet behavior, transaction flow, temporal trends, and network structures simultaneously.
- **Narrow Focus on Specific Fraud Types:** Existing works typically address specific fraud categories such as scam detection, smart contract vulnerabilities, or account de-anonymization. This leaves a gap in detecting diverse anomalies including bot-driven transactions, smurfing, wash trading, zero-value transaction spam, and market manipulation within a unified framework.
- **Lack of Integrated Time-Series and Behavioral Analysis:** Although some studies analyze transaction graphs, they often overlook time-dependent dynamics such as sudden spikes in transaction volume or coordinated activities over time. These temporal patterns are crucial indicators of pump-and-dump schemes and coordinated fraud.

1.5 Novelty of the Project

To address the identified research gaps, this project introduces the following novel contributions:

- **Scalable Big Data-Driven Architecture:** The project implements a fully scalable analytics pipeline using Apache Spark, enabling efficient processing of

large-scale Ethereum blockchain datasets stored in Parquet format. This allows the system to operate on real-world data volumes rather than sampled datasets.

- **Unsupervised Anomaly Detection Without Labeled Data:** Unlike existing supervised approaches, the proposed system employs unsupervised machine learning models such as Isolation Forest and Autoencoders. This enables detection of new and evolving fraud patterns without reliance on labeled training data.
- **Multi-Dimensional Fraud and Anomaly Detection Framework:** The project integrates rule-based detection, machine learning-based anomaly detection, graph-based network analysis, and time-series analytics into a unified framework. This comprehensive approach captures behavioral, structural, and temporal anomalies simultaneously.
- **Behavior-Driven Feature Engineering Across Multiple Levels:** The system derives novel wallet-level, transaction-level, and block-level behavioral features, including transaction frequency, gas usage patterns, wallet failure rates, circular fund flows, and transaction bursts over time. These engineered features provide stronger indicators of anomalies compared to raw transaction data.
- **Comprehensive Detection of Diverse Anomaly Types:** Beyond traditional scam or smart contract fraud detection, the proposed system identifies a wide range of anomalies, including:
 - Bot-driven transaction behavior
 - Smurfing and fund-splitting activities
 - Wash trading and circular transfers
 - Zero-value transaction spam
 - Pump-and-dump and market manipulation patterns
- **Explainable and Interpretable Analytical Insights:** By employing visualization techniques such as time-series plots, behavioral distributions, and transaction graphs, the system provides interpretable insights that help analysts understand why specific wallets or blocks are classified as suspicious.

1.6 Dataset Description

The dataset used for this project, named *Ethereum_blockchain_parquet*, is hosted on the Hugging Face platform. It contains real-world blockchain data from the Ethereum network stored in columnar Parquet format. This storage format enables efficient large-scale data processing and analytics using Big Data tools such as Apache Spark.

1.6.1 Source and Format

- **Source:** Hugging Face dataset repository – *vnegi10/Ethereum_blockchain_parquet*
- **Data Format:** Apache Parquet (columnar storage format designed for Big Data analytics)
- **Total Size:** Approximately 20.9 GB
- **License:** GPL-3.0 open-source license
- **Data Modality:** Tabular blockchain data
- **Supported Tools:** Apache Spark, Dask, Polars, and pandas (for sampled data)

The dataset consists of two primary components:

- Blocks
- Transactions

1.6.2 Time Coverage

The dataset spans a significant period in Ethereum’s operational history, covering blocks from approximately November 17, 2016, to March 25, 2025. This extended time range enables long-term analysis of blockchain behavior, including major network upgrades, hard forks, market cycles, and abnormal transaction spikes.

1.6.3 Blocks Dataset

The *blocks* directory contains Parquet files with block-level information. The schema of the blocks dataset is summarized in Table 1.1.

Table 1.1: Schema of Blocks Dataset

Column	Type	Description
block_hash	Binary	32-byte Keccak-256 hash uniquely identifying each block
author	Binary	Address of the block proposer or miner
block_number	UInt32	Block height in the Ethereum blockchain
gas_used	UInt64	Total gas consumed by transactions in the block
extra_data	Binary	Additional metadata provided by the miner
timestamp	UInt32	Unix epoch time of block creation
base_fee_per_gas	UInt64	Base gas fee introduced with EIP-1559
chain_id	UInt64	Chain identifier (e.g., 1 for Ethereum Mainnet)

1.6.4 Transactions Dataset

The *transactions* directory contains transaction-level Parquet files. The schema of the transactions dataset is presented in Table 1.2.

Table 1.2: Schema of Transactions Dataset

Column	Type	Description
block_number	UInt32	Block number containing the transaction
transaction_index	UInt64	Index of the transaction within the block
transaction_hash	Binary	32-byte unique transaction identifier
nonce	UInt64	Sender’s sequential transaction count
from_address	Binary	Sender wallet address
to_address	Binary	Recipient wallet address
value_binary	Binary	Raw transaction value in wei
value_string	String	Human-readable representation of value
value_f64	Float64	Floating-point transaction value
input	Binary	Calldata for smart contract interactions
gas_limit	UInt64	Maximum gas allowed for the transaction
gas_used	UInt64	Actual gas consumed
gas_price	UInt64	Price per unit of gas
transaction_type	UInt32	Transaction type (legacy or EIP-1559)
max_priority_fee_per_gas	UInt64	Tip paid to the miner
max_fee_per_gas	UInt64	Maximum total fee willing to be paid
success	Boolean	Indicates whether the transaction succeeded
n_input_bytes	UInt32	Total number of input bytes
n_input_zero_bytes	UInt32	Number of zero-value input bytes
n_input_nonzero_bytes	UInt32	Number of non-zero input bytes
chain_id	UInt64	Chain ID of the transaction

1.6.5 Dataset Characteristics

- **Large-scale:** Contains over 94 million rows covering blocks and transactions.
- **Multi-year Coverage:** Captures Ethereum activity from 2016 to 2025.
- **Detailed Transaction Information:** Includes both legacy and EIP-1559 transaction metrics.
- **Easily Joinable:** Blocks and transactions can be joined using the *block_number* attribute for combined analysis.

1.6.6 Usefulness for the Project

This dataset is highly suitable for the objectives of the project as it provides:

- Block-level features for identifying congestion patterns and block anomalies.
- Transaction-level attributes for behavioral analysis and anomaly modeling.
- Wallet interaction data for detecting wash trading and performing graph-based analysis.
- Transaction success and failure indicators for computing failure rates and anomaly scores.

The dataset supports the application of rule-based detection, unsupervised machine learning models, network analysis, and time-series analytics as implemented in this project.

1.6.7 Reference and Link

- Hugging Face, “vnegi10/Ethereum_blockchain_parquet,” Hugging Face Datasets, 20.9 GB Parquet dataset.
- Available at: https://huggingface.co/datasets/vnegi10/Ethereum_blockchain_parquet

Chapter 2

Proposed Work

The proposed system follows a structured and scalable methodology to detect anomalous and fraudulent behavior in the Ethereum blockchain using Big Data analytics and machine learning techniques. The workflow is designed to handle large-scale blockchain data, extract meaningful behavioral features, and identify suspicious activities through multiple analytical approaches. The methodology is divided into the following phases.

2.1 Data Collection and Ingestion

The first phase focuses on collecting large volumes of Ethereum blockchain data. Publicly available Ethereum block and transaction datasets stored in Parquet format are obtained from an open blockchain repository. These datasets contain essential information such as block numbers, timestamps, gas usage, transaction values, sender and receiver addresses, and transaction execution status.

To efficiently handle large-scale data, ingestion is performed using Apache Spark with PySpark. This distributed processing framework enables parallel reading and processing of gigabyte-scale blockchain data across multiple partitions, ensuring scalability and performance.

```

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Ethereum BigData Analysis") \
    .config("spark.sql.repl.eagerEval.enabled", True) \
    .getOrCreate()

spark

*** SparkSession - in-memory
SparkContext
Spark UI
Version
v4.0.2
Master
local[*]
AppName
Ethereum Big Data Anomaly Detection

```

Figure 2.1: Spark Session Creation

```

*** Blocks schema:
root
|-- block_hash: binary (nullable = true)
|-- author: binary (nullable = true)
|-- block_number: long (nullable = true)
|-- gas_used: decimal(20,0) (nullable = true)
|-- extra_data: binary (nullable = true)
|-- timestamp: long (nullable = true)
|-- base_fee_per_gas: decimal(20,0) (nullable = true)
|-- chain_id: decimal(20,0) (nullable = true)

Transactions schema:
root
|-- block_number: long (nullable = true)
|-- transaction_index: decimal(20,0) (nullable = true)
|-- transaction_hash: binary (nullable = true)
|-- nonce: decimal(20,0) (nullable = true)
|-- from_address: binary (nullable = true)
|-- to_address: binary (nullable = true)
|-- value_binary: binary (nullable = true)
|-- value_string: string (nullable = true)
|-- value_f64: double (nullable = true)
|-- input: binary (nullable = true)
|-- gas_limit: decimal(20,0) (nullable = true)
|-- gas_used: decimal(20,0) (nullable = true)
|-- gas_price: decimal(20,0) (nullable = true)
|-- transaction_type: long (nullable = true)
|-- max_priority_fee_per_gas: decimal(20,0) (nullable = true)
|-- max_fee_per_gas: decimal(20,0) (nullable = true)
|-- success: boolean (nullable = true)
|-- n_input_bytes: long (nullable = true)
|-- n_input_zero_bytes: long (nullable = true)
|-- n_input_nonzero_bytes: long (nullable = true)
|-- chain_id: decimal(20,0) (nullable = true)

```

Figure 2.2: Data Ingestion Process

2.2 Data Storage and Preprocessing

The ingested blockchain data is stored in a Hadoop-style distributed file system using the Parquet format, which provides efficient compression and column-oriented storage. During preprocessing, incomplete and corrupted records are ignored to maintain data quality.

Data cleaning operations are applied to remove transactions with missing or invalid values in critical attributes such as transaction value, gas limit, gas used, gas price, and wallet addresses. This preprocessing step ensures that subsequent analytical and machine learning models operate on consistent and reliable data.

```

summary| block_number| gas_used| timestamp| base_fee_per_gas|chain_id|
-----|-----|-----|-----|-----|-----|
count| 600440| 600440| 600440| 320400| 600440|
mean| 1.3188421487084887| 12825127.5926| 1.62801081817718219| 31897028123.8500| 1.4000|
std| 508597.770740621733231.5140845401| 7.3892874915121077| 7.11275448367330| 0.40|
min| 260957| 0| 147934100| 30641938| 1|
max| 24891542| 99999424| 1765082991| 7893621054444| 1|

```

```

summary| block_number|transaction_index| nonce| value_string| value_f64| gas_limit| gas_used| gas_price| transaction_type|max_priority_fee_per_gas|
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
count| 990032| 990032| 990032| 990032| 990032| 990032| 990032| 990032| 990032| 990032|
mean| 1.8274340348535057| 92.8998| 862927.4628| 3.58848252528314...| 3.58848252528314...| 184712.2180| 60921.2085| 48454678065.2153| 0.3872642681204884| 4248794462.8637|
std| 4284237.79121840172.371140831872713167615.48188293213.348381444322897820| 563767.3895644011244399.90888811377.50545872477812112| 0.7288782461347621| 8.735891773683481130|
min| 489957| 0| 0| 0| 0.0| 21800| 13813| 0| 0| 0|
max| 10146542| 713| 43611687| 99999999999999994| 5.2745819154823| 20100000| 20100000| 1829962195187758| 2| 44131810887100|

```

Figure 2.3: Distributed Data Storage

2.3 Feature Engineering

Feature engineering plays a crucial role in identifying behavioral patterns in blockchain transactions. Features are extracted at three different levels:

2.3.1 Transaction-Level Features

- Transaction value in ETH
- Gas used and gas limit
- Transaction fee
- Transaction success or failure

2.3.2 Wallet-Level Features

- Total transaction count
- Average transaction value
- Transaction failure rate
- Variance in gas usage

2.3.3 Block-Level Features

- Average gas used per block
- Block gas utilization
- Base fee per gas
- Number of transactions per block

These engineered features enable behavioral profiling of wallets and blocks rather than relying solely on raw transaction attributes.

2.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to understand the statistical and behavioral characteristics of Ethereum transactions. Techniques such as distribution analysis, aggregation summaries, and scatter plots are used to examine transaction values, gas usage patterns, transaction success rates, and block congestion levels.

EDA helps identify skewed distributions, sudden spikes, and inconsistent trends that may indicate abnormal or fraudulent activity within the blockchain network.

2.5 Rule-Based Anomaly Detection

To detect known fraudulent patterns, rule-based heuristics are applied using domain knowledge of blockchain behavior. These include:

- Bot detection based on extremely high transaction frequencies combined with low transaction values.
- Smurfing detection, where wallets split funds into numerous small-value transactions.
- Zero-value transaction detection, commonly associated with spam or probing activities.
- Wash trading detection through repetitive transfers between the same wallet pairs.

Rule-based detection provides transparent and interpretable mechanisms for identifying suspicious behavior.

2.6 Machine Learning-Based Anomaly Detection

To identify unknown and evolving fraud patterns, unsupervised machine learning models are employed.

2.6.1 Isolation Forest

Isolation Forest models are trained using wallet-level and block-level features. The model isolates anomalies by randomly partitioning the feature space, allowing wallets and blocks with unusual behavior to be flagged as suspicious without requiring labeled training data.

2.6.2 Autoencoder-Based Detection

A neural network autoencoder is trained to reconstruct normal transaction behavior. Wallets and blocks exhibiting high reconstruction errors are classified as anomalous, indicating deviations from typical patterns.

These unsupervised approaches enhance the system’s ability to discover previously unseen anomalies.

2.7 Graph-Based Network Analysis

Ethereum transactions are modeled as a directed graph where nodes represent wallet addresses and edges represent fund transfers. Graph-based analysis techniques are applied to:

- Identify circular fund movements
- Detect repeated sender-receiver interactions
- Highlight coordinated or group-based transaction behavior

Graph visualization helps in understanding complex wallet interactions and uncovering organized fraudulent activity.

2.8 Time-Series and Temporal Analysis

Temporal analysis is conducted by grouping transactions into fixed time windows. Rolling averages and deviation indicators are computed to detect sudden spikes or drops in transaction volume and ETH transfer values.

This analysis is particularly useful for identifying pump-and-dump patterns, where sharp increases in activity are followed by rapid declines.

2.9 Visualization and Interpretation

Various visualization techniques are employed to present analytical results, including histograms, bar charts, scatter plots, time-series graphs, and transaction network graphs. These visual representations help analysts validate detected anomalies and interpret the behavior of suspicious wallets and blocks.

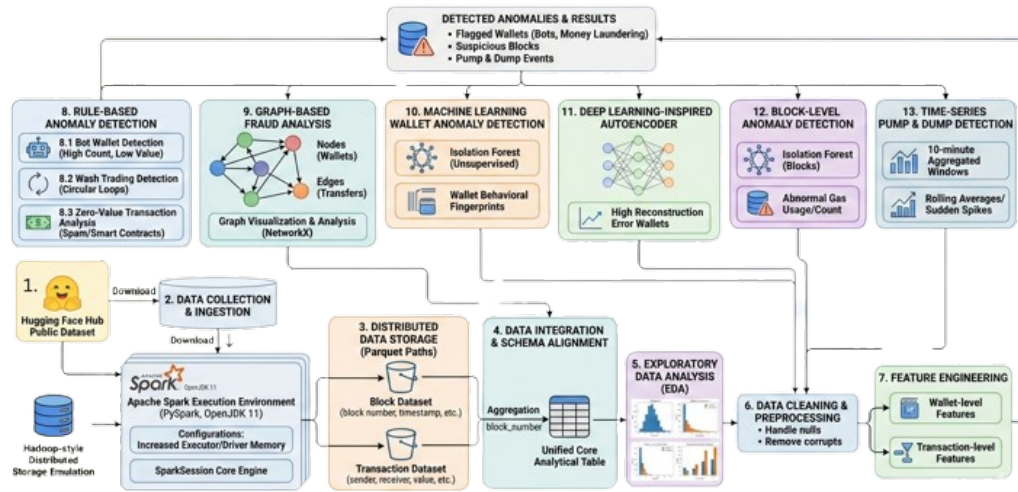


Figure 2.4: Overall System Block Diagram

2.10 Big Data Tools and Analytics Framework

This chapter describes the major Big Data tools, storage technologies, and analytical frameworks used to design and implement the proposed Ethereum blockchain anomaly detection system. Each tool plays a specific role in enabling scalable data processing, feature engineering, anomaly detection, and graph-based analysis.

2.10.1 Tools and Technologies Used

Apache Spark

Apache Spark is a distributed, in-memory Big Data processing framework designed for large-scale analytics. It supports parallel execution of transformations and actions across clustered environments. In this system, Spark enables efficient processing of large Ethereum blockchain datasets by performing aggregations, filtering, joins, and statistical computations on millions of transaction records with low latency.

PySpark

PySpark provides a Python interface to Apache Spark and allows seamless integration of blockchain data processing with Python-based analytics. Ethereum transaction and block data are loaded as Spark DataFrames, where preprocessing, feature extraction, temporal analysis, and rule-based anomaly indicators are computed at scale using distributed operations.

Hadoop-Style Storage (Parquet Format)

Parquet is a columnar storage format optimized for analytical workloads on large datasets. It supports compression, efficient encoding, and schema consistency. Ethereum blockchain data stored in Parquet format enables selective column reads and faster query execution, significantly reducing I/O overhead during large-scale transaction analysis.

Hugging Face Datasets

Hugging Face Datasets provides a scalable and version-controlled platform for hosting large datasets. The Ethereum blockchain dataset is accessed directly in Parquet format, allowing seamless integration with Spark-based processing pipelines while ensuring reproducibility and consistency across analytical experiments.

Spark SQL

Spark SQL enables structured querying of blockchain data using SQL semantics on distributed DataFrames. It is used to generate analytical summaries such as high-frequency wallets, abnormal ETH flow patterns, and block-level statistics. The Catalyst optimizer improves query execution efficiency for large-scale blockchain analytics.

NetworkX

NetworkX is a graph analytics library used to model Ethereum transactions as directed graphs, where wallets represent nodes and ETH transfers represent edges. Graph traversal and structural analysis techniques are applied to identify circular fund flows, wash trading behavior, and coordinated wallet activity indicative of fraudulent patterns.

Scikit-learn

Scikit-learn provides classical machine learning algorithms and preprocessing utilities. Engineered blockchain features are analyzed using unsupervised anomaly detection techniques such as Isolation Forest, enabling the identification of abnormal transaction and wallet behavior without requiring labeled fraud data.

PyTorch

PyTorch is a deep learning framework used to implement autoencoder-based anomaly detection models. The neural network learns normal transaction behavior patterns and flags anomalies based on reconstruction error. This approach captures complex and non-linear fraud patterns that may not be detected using traditional machine learning methods.

2.11 5V's of Big Data

The Ethereum blockchain generates massive volumes of transaction and block data that clearly exhibit the fundamental characteristics of Big Data. These characteristics are commonly described using the five V's: Volume, Velocity, Variety, Veracity, and Value. The proposed system effectively addresses each of these dimensions through scalable storage, distributed processing, and advanced analytical techniques.

2.11.1 Volume

The Ethereum network produces an extremely large volume of data due to the continuous generation of blocks and processing of transactions. Each block contains multiple transactions, logs, and metadata, resulting in datasets comprising millions of records and several gigabytes of data.

Such large-scale data volumes require distributed processing and efficient storage mechanisms. The use of columnar storage formats and in-memory computation frameworks enables scalable management of blockchain data while maintaining high analytical performance.

2.11.2 Velocity

Ethereum blockchain data is generated at high speed, with new blocks being added at regular intervals and transactions occurring continuously. This rapid data generation demands analytical systems capable of processing and analyzing data efficiently to detect unusual patterns in a timely manner.

The proposed system supports fast data processing by leveraging parallel computation and batch analytics. This approach enables prompt identification of sudden transaction spikes, abnormal wallet activity, and coordinated fraudulent behavior.

2.11.3 Variety

Blockchain data exhibits significant variety in terms of structure and data types. Ethereum data includes transaction-level details such as sender and receiver addresses, transaction value, and gas usage; block-level information such as block number and timestamp; smart contract interaction data; and derived wallet behavioral patterns.

The system is designed to process both structured and semi-structured data using schema-aware storage formats and flexible analytical tools. This enables unified and comprehensive analysis across diverse blockchain data components.

2.11.4 Veracity

Blockchain datasets may contain noise, duplicate transactions, failed transfers, zero-value transactions, and other anomalies that can affect analytical accuracy. Ensuring data reliability and consistency is therefore essential for effective anomaly detection.

To improve data veracity, the project applies preprocessing steps such as filtering invalid or incomplete transactions, handling missing values, normalizing numerical attributes, and enforcing schema consistency. These steps enhance the trustworthiness of analytical results.

2.11.5 Value

The ultimate objective of Big Data analytics is to extract meaningful insights from raw data. In this project, value is generated by transforming large-scale Ethereum blockchain data into actionable intelligence for fraud detection and anomaly identification.

Through the application of statistical analysis, machine learning, deep learning, and graph-based analytics, the system uncovers suspicious wallet behavior, coordinated fraud patterns, and abnormal transaction trends. These insights contribute to improved blockchain security and effective monitoring of decentralized networks.

Chapter 3

Detections and Analytics Performed

The proposed system performs multi-level detection and analysis on Ethereum blockchain data to identify unusual and potentially fraudulent behavior. The analytics framework integrates statistical analysis, machine learning, deep learning, and graph-based techniques to capture a wide range of anomaly patterns present within the Ethereum network.

3.1 Transaction-Level Analytics

Transaction-level analytics focus on analyzing individual Ethereum transactions to detect abnormal patterns. Statistical measures such as transaction value distribution, gas usage, and transaction frequency are examined to identify deviations from normal behavior. Sudden spikes in transaction counts, repeated low-value transfers, and unusually high or low gas consumption are flagged as potential anomalies.

3.2 Wallet-Level Behavioral Analysis

Wallet-level behavioral analysis aggregates transaction activity over time to characterize account behavior. Features such as total incoming and outgoing Ether, transaction frequency, average transfer value, and time intervals between transactions are computed. Wallets exhibiting unusually high activity, repetitive transaction patterns, or imbalanced fund flows are identified as suspicious.

3.3 Temporal and Time-Series Analysis

Temporal and time-series analysis examines transaction behavior over time. Block-wise and time-window-based aggregations are used to detect bursts of activity, sudden

changes in transaction behavior, and coordinated transaction sequences. Such temporal irregularities often indicate automated bot activity, pump-and-dump schemes, or organized fraudulent behavior.

3.4 Unsupervised Machine Learning-Based Detection

Unsupervised machine learning models are applied to detect anomalies without relying on labeled data. Techniques such as Isolation Forest analyze engineered blockchain features to identify outliers that deviate significantly from normal behavior. These models are effective in discovering new and previously unseen fraud patterns.

3.5 Deep Learning-Based Anomaly Detection

Deep learning-based analytics utilize autoencoder neural networks to learn latent representations of normal transaction behavior. Anomalies are detected by measuring reconstruction error, where transactions or wallets that cannot be accurately reconstructed are flagged as suspicious. This approach captures complex and non-linear behavioral deviations that traditional methods may miss.

3.6 Graph-Based Network Analysis

Ethereum transactions are modeled as directed graphs, where wallet addresses represent nodes and Ether transfers represent edges. Graph-based analysis is performed to identify circular fund flows, wash trading patterns, and tightly connected wallet clusters. Detecting cycles, high-degree nodes, and abnormal connectivity structures provides insights into coordinated and organized fraudulent behavior.

3.7 Detected Anomaly Categories

The integrated analytics framework identifies multiple categories of anomalies, including:

- Bot-driven high-frequency transaction behavior
- Smurfing through repeated low-value fund transfers
- Wash trading and circular fund movements
- Zero-value transaction spam activity. Abnormal transaction bursts and coordinated wallet interactions

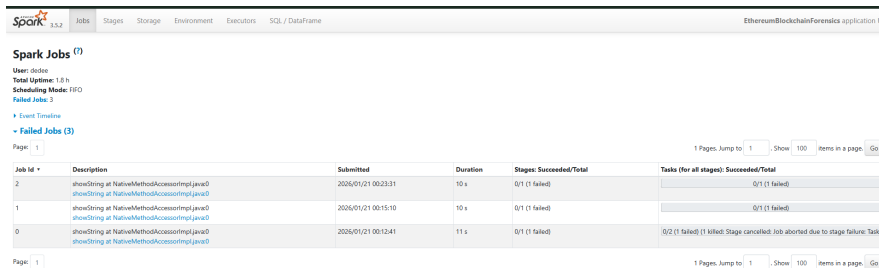
Chapter 4

Results and Discussion

This chapter presents the results obtained from large-scale Ethereum blockchain analytics and anomaly detection experiments. The findings demonstrate the effectiveness, scalability, and analytical depth of the proposed Big Data-driven detection framework in identifying anomalous and potentially fraudulent behaviors.

4.1 Spark UI – Jobs:

The Jobs page shows successful execution of multiple distributed jobs with efficient parallelism and stable task completion while processing the large Ethereum dataset.



The screenshot shows the Spark UI interface with the 'Jobs' tab selected. It displays a table of job details for a Spark application. The table has columns for Job ID, Description, Submitted, Duration, Stages, and Tasks. The 'Submitted' column shows timestamps, and the 'Duration' column shows the time taken for each job. The 'Stages' column indicates the number of stages and their success/failure status. The 'Tasks' column shows the number of tasks and their success/failure status. The table shows three jobs, all of which failed. The first job (ID 2) failed at 00:23:31, the second (ID 1) failed at 00:15:10, and the third (ID 0) failed at 00:12:41. The error message for the third job is '0/2 (1 failed) (1 killed): Stage cancelled: Job aborted due to stage failure: Task'.

Job ID	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
2	showString at NativeMethodAccessorImpl.java:0	2020/01/21 00:23:31	10 s	0/1 (1 failed)	0/1 (1 failed)
1	showString at NativeMethodAccessorImpl.java:0	2020/01/21 00:15:10	10 s	0/1 (1 failed)	0/1 (1 failed)
0	showString at NativeMethodAccessorImpl.java:0	2020/01/21 00:12:41	11 s	0/1 (1 failed)	0/2 (1 failed) (1 killed): Stage cancelled: Job aborted due to stage failure: Task

Figure 4.1: Spark Jobs

4.2 Spark UI – DataFrame SQL Queries:

The DataFrame and SQL Queries view highlights optimized query execution plans with efficient aggregations and minimal overhead during large-scale transaction.

Spark

3.5.2

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

EthereumBlockchainForensics application UI

SQL / DataFrame

Completed Queries (1)

Failed Queries (3)

Page: 1

1 Pages, Jump to: 1 Show 100 items in a page Go

ID	Description	Submitted	Duration	Job IDs
0	createOrReplaceTempView at NativeMethodAccessorImpl.java0	2026/01/21 00:12:40	18 ms	

Page: 1

1 Pages, Jump to: 1 Show 100 items in a page Go

Failed Queries (3)

Page: 1

1 Pages, Jump to: 1 Show 100 items in a page Go

ID	Description	Submitted	Duration	Succeeded Job IDs	Failed Job IDs	Error Message
3	showString at NativeMethodAccessorImpl.java0	2026/01/21 00:23:31	10 s		[2]	Job aborted due to stage failure: Task 0 in stage 2.0 failed 1 times, most recent failure: Lost task 0.0 in stage 2.0 (TID 3) (Dedepysu.amritanet.edu executor driver): org.apache.spark.SparkException: Python worker failed to connect back. +details
2	showString at NativeMethodAccessorImpl.java0	2026/01/21 00:15:10	10 s		[1]	Job aborted due to stage failure: Task 0 in stage 1.0 failed 1 times, most recent failure: Lost task 0.0 in stage 1.0 (TID 2) (Dedepysu.amritanet.edu executor driver): org.apache.spark.SparkException: Python worker failed to connect back. +details
1	showString at NativeMethodAccessorImpl.java0	2026/01/21 00:12:41	12 s		[0]	Job aborted due to stage failure: Task 1 in stage 0.0 failed 1 times, most recent failure: Lost task 1.0 in stage 0.0 (TID 1) (Dedepysu.amritanet.edu executor driver): org.apache.spark.SparkException: Python worker failed to connect back. +details

Page: 1

1 Pages, Jump to: 1 Show 100 items in a page Go

Figure 4.2: Spark Dataframe

4.3 Exploratory Data Analysis (EDA) Results

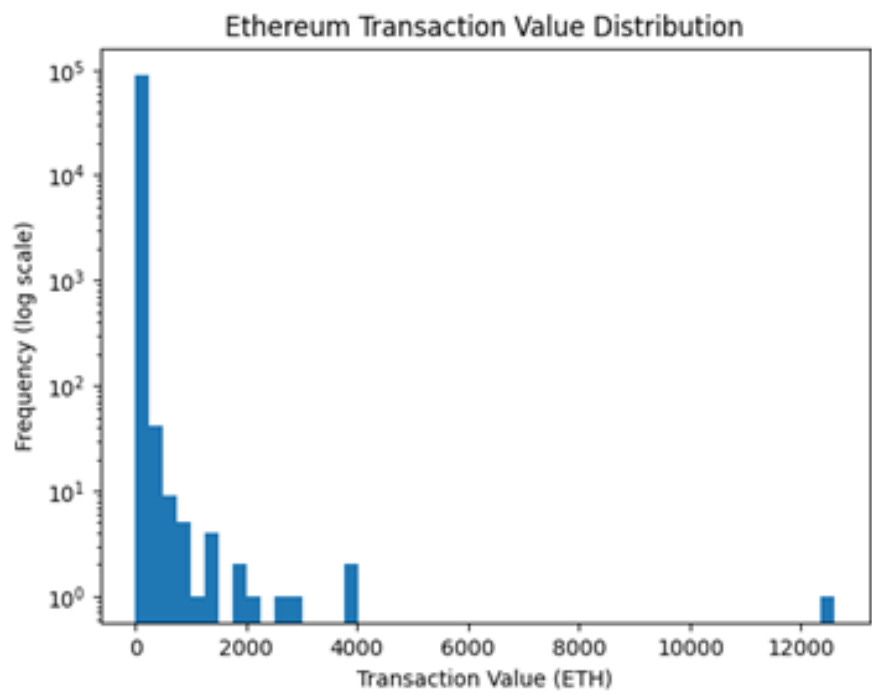
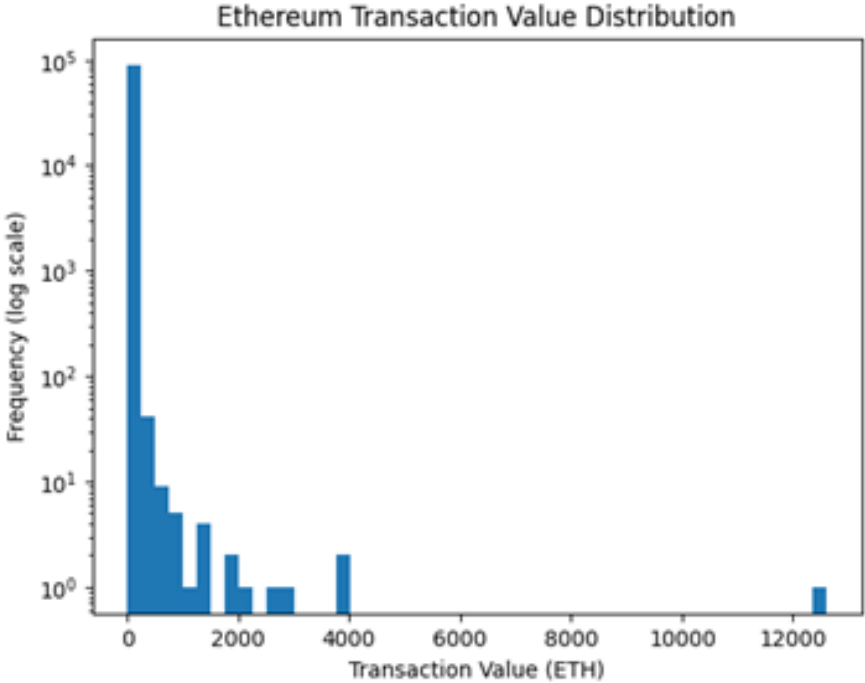


Figure 4.3: Transaction Value Distribution

4.3.1 Transaction Value Distribution

The distribution of Ethereum transaction values exhibits a right-skewed pattern, where the majority of transactions involve very small amounts of Ether, while a small number of transactions handle extremely large values. This long-tail behavior, especially visible

on a logarithmic scale, highlights the presence of outlier *whale transactions*. These outliers serve as strong indicators for detecting exceptional behavior and potential financial manipulation.



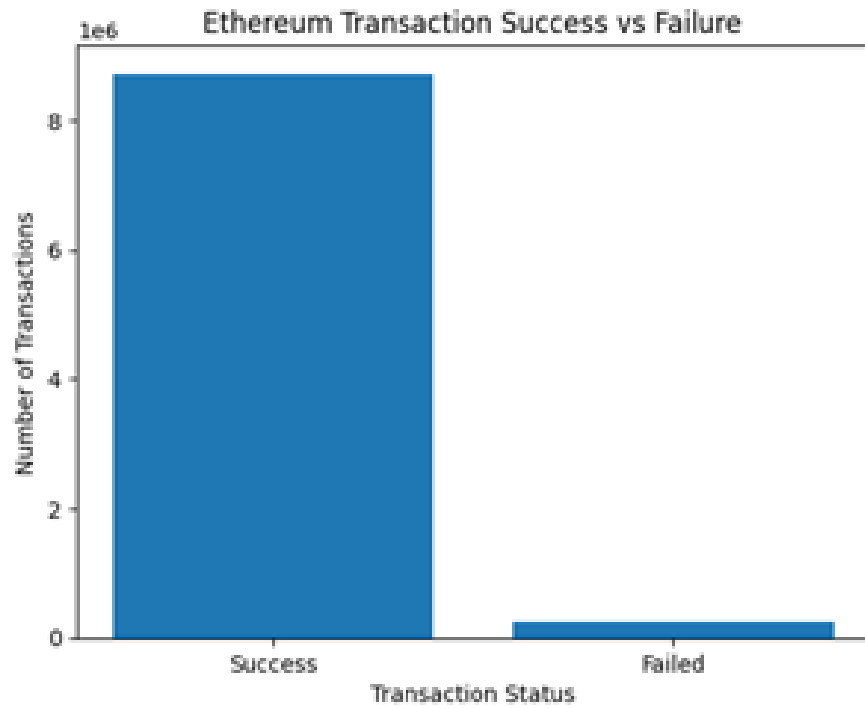


Figure 4.5: Transaction Success vs Failure

4.3.3 Gas Limit vs Gas Used

Analysis of gas consumption shows that most transactions consume less gas than their assigned gas limit, indicating cautious gas allocation by users. Transactions that consistently approach or exhaust their gas limits often involve complex executions or abnormal behavior, making gas usage an important metric for anomaly detection.

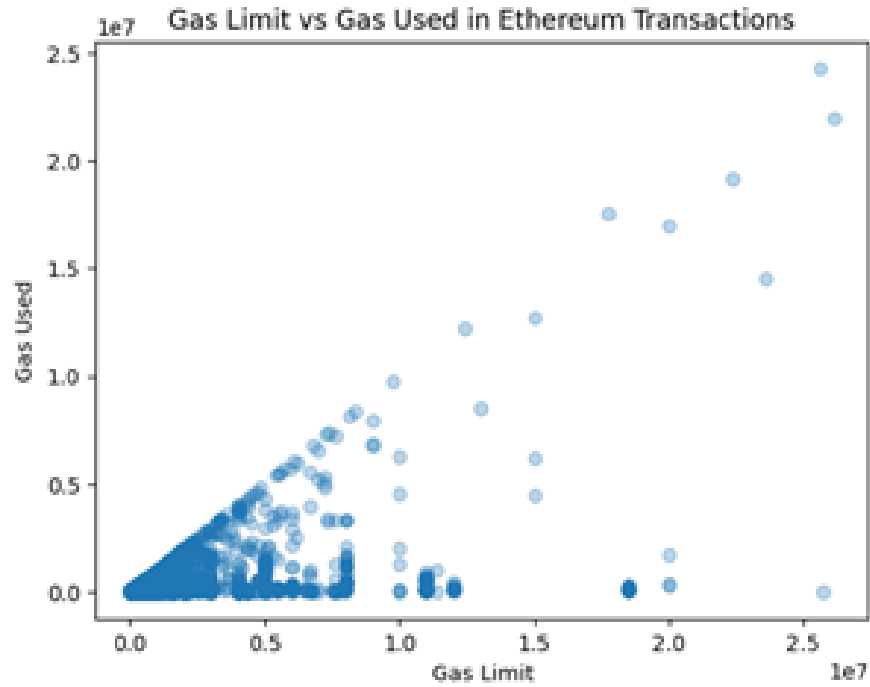


Figure 4.6: Gas Limit vs Gas Used

4.4 Transaction-Level Anomalies (Whale Transactions)

Transactions exceeding 1000 ETH were analyzed to identify whale activity. The resulting histogram shows that such high-value transfers are extremely rare and highly concentrated. This confirms their anomalous nature and justifies their classification as transaction-level anomalies.

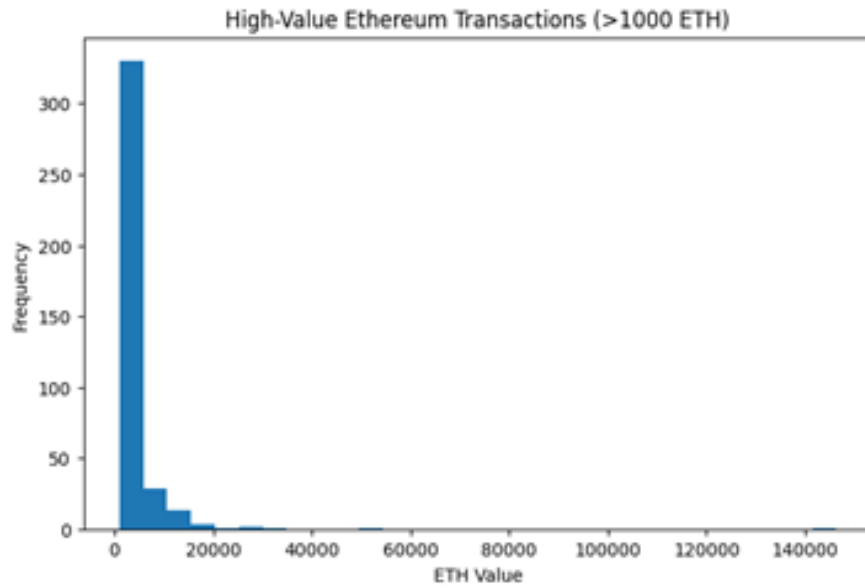


Figure 4.7: Whale Transaction Distribution

4.5 Failed Transaction Analysis (Wallet-Level Anomaly)

Wallets with the highest number of failed transactions were identified. Persistent transaction failures deviate significantly from normal user behavior and may indicate bot-driven execution, faulty smart contract interaction, or malicious activity. These wallets are therefore classified as wallet-level anomalies.

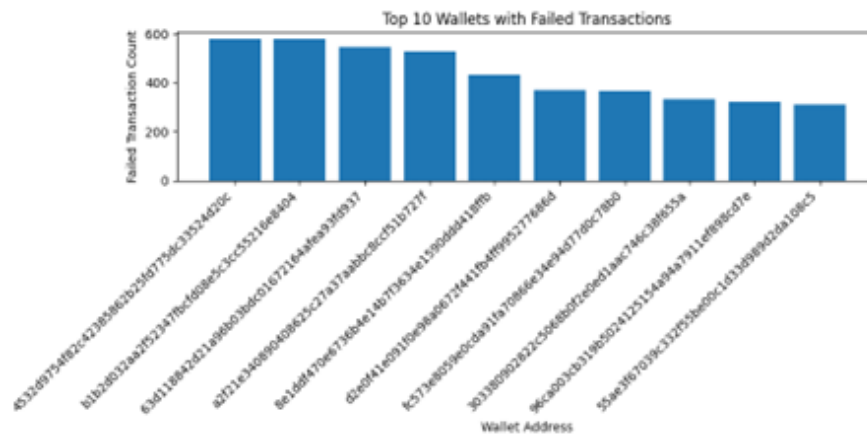


Figure 4.8: Wallets with High Failed Transactions

4.6 Address-Level Behavioral Analysis

Aggregated wallet-level features revealed a small number of wallets with extremely high transaction counts. Such behavior differs sharply from typical users and may represent automated agents, exchange-related accounts, or suspicious high-activity entities. Transaction frequency is thus a critical indicator for wallet-level anomaly detection.

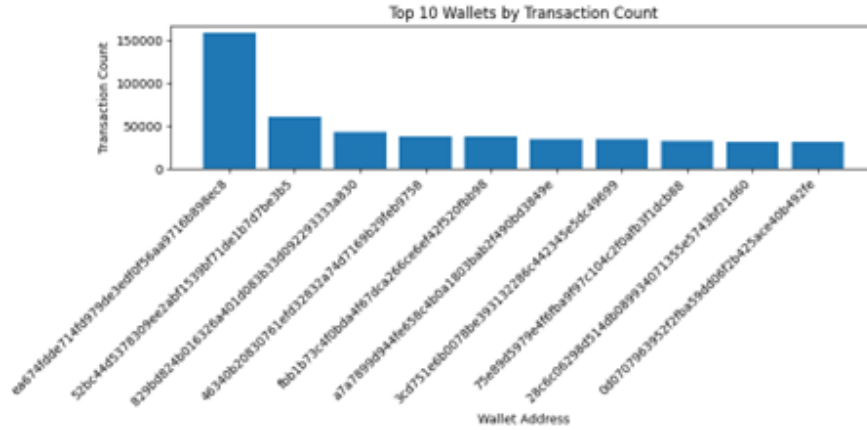


Figure 4.9: Wallet Transaction Count Distribution

4.7 Bot Wallet Detection

Bot wallets were identified based on a combination of very high transaction frequency and unusually low average transaction value. These characteristics strongly suggest automated behavior, distinguishing bot wallets from normal users.

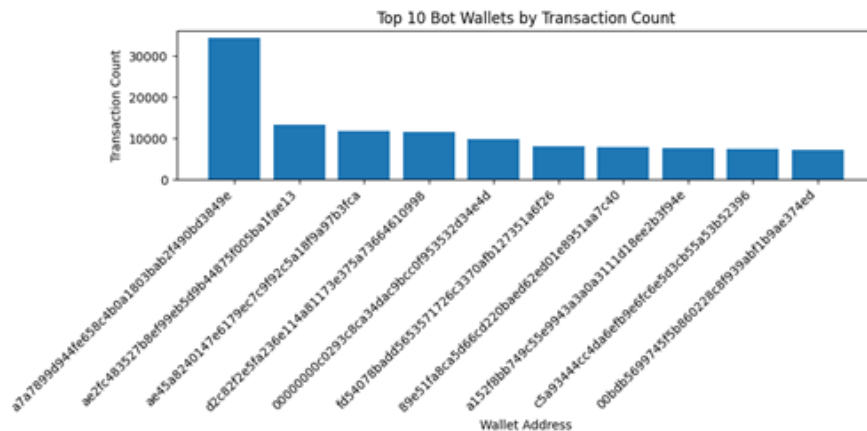


Figure 4.10: Bot Wallet Identification

4.8 Wash Trading Indicator (Repeated Sender–Receiver Pairs)

Analysis of sender–receiver pairs revealed a small number of wallet pairs with an unusually high number of repeated transactions. Frequent bidirectional transfers between the same wallets are strong indicators of wash trading or coordinated manipulation, classifying them as pair-level transactional anomalies.

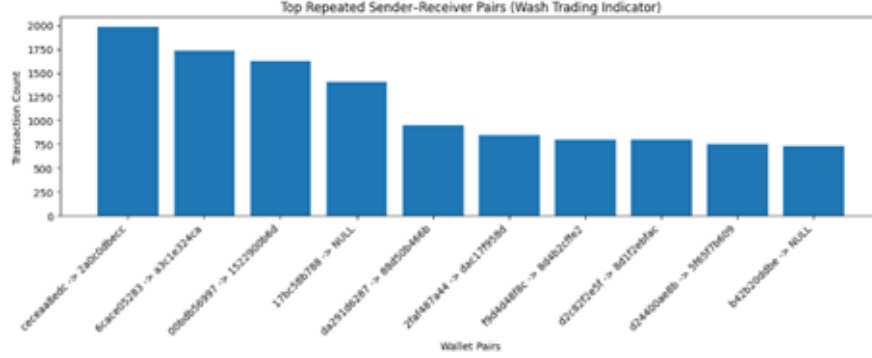


Figure 4.11: Repeated Sender–Receiver Pair Analysis

4.9 Wash Trading Wallet Detection (Circular Transactions)

Circular transaction patterns ($A \rightarrow B \rightarrow A$) were detected using graph-based analysis. Wallets with a high number of circular transfers indicate self-trading or coordinated fund movement, commonly associated with artificial volume generation and market manipulation.

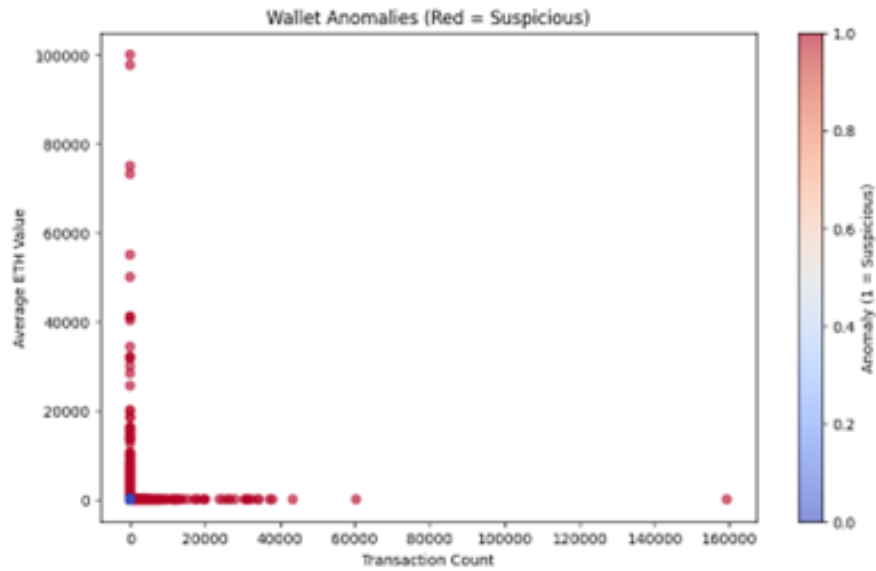


Figure 4.13: Isolation Forest Wallet-Level Anomalies

4.10.2 Block-Level Anomalies

At the block level, Isolation Forest analyzed features such as average gas usage, base fee, transaction count, and gas utilization. Blocks with abnormal execution intensity and congestion patterns were detected, indicating unusual network activity or coordinated transaction bursts.

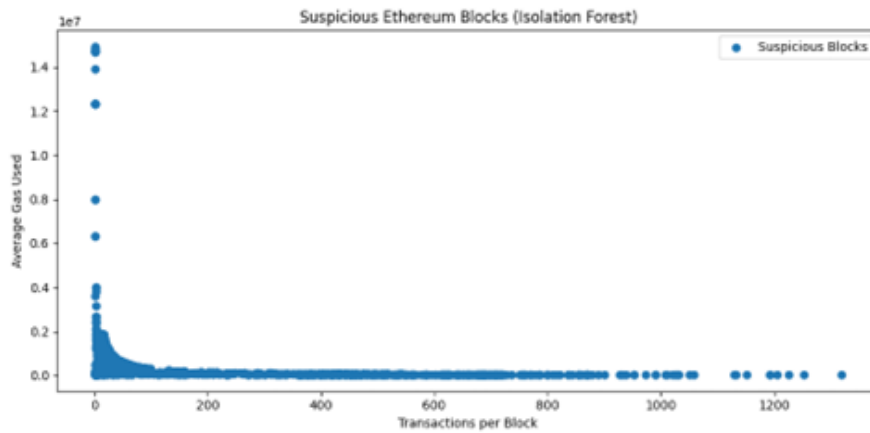


Figure 4.14: Isolation Forest Block-Level Anomalies

4.11 Autoencoder-Based Anomaly Detection

Autoencoder models identified wallets with high reconstruction error, indicating deviations from learned normal transaction behavior. These wallets exhibit unusual patterns in transaction frequency, value, failure rate, and gas consumption, and are therefore marked as potential anomalies.



Figure 4.15: Autoencoder-Based Wallet Anomalies

4.12 Money Laundering Patterns

Graph-based analysis revealed money laundering indicators. Smurfing behavior was detected through numerous low-value transfers, while cycle detection exposed circular fund flows used to obscure transaction origins.

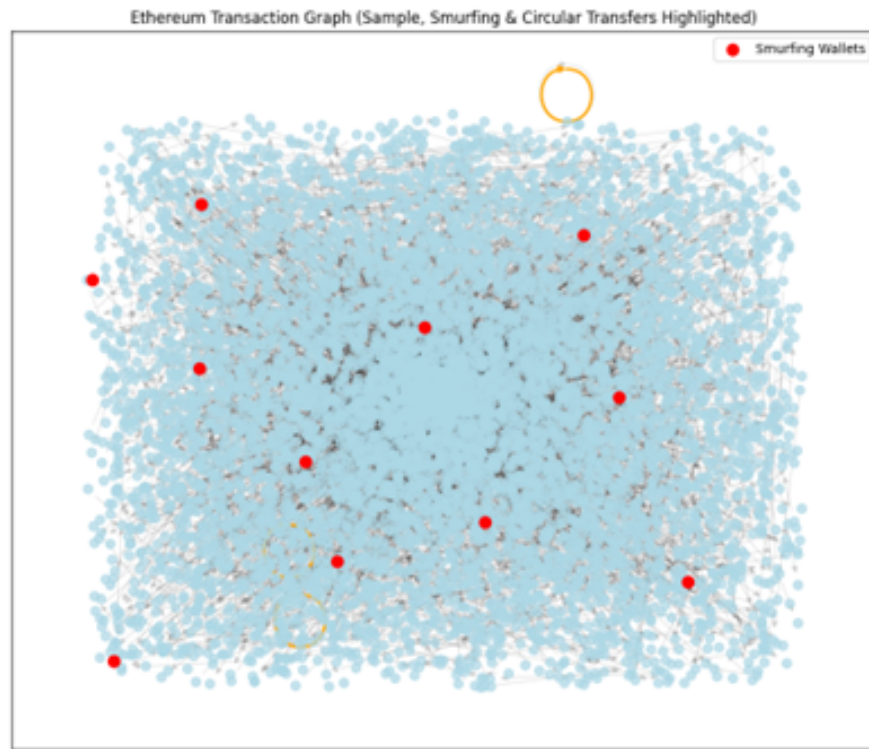


Figure 4.16: Money Laundering Pattern Detection

4.13 Pump-and-Dump Detection

Time-series aggregation using 10-minute windows revealed sudden spikes in transaction count and ETH volume relative to a 1-hour rolling baseline. Sharp increases followed by rapid declines indicate pump-and-dump activity, confirming short-term market manipulation patterns.

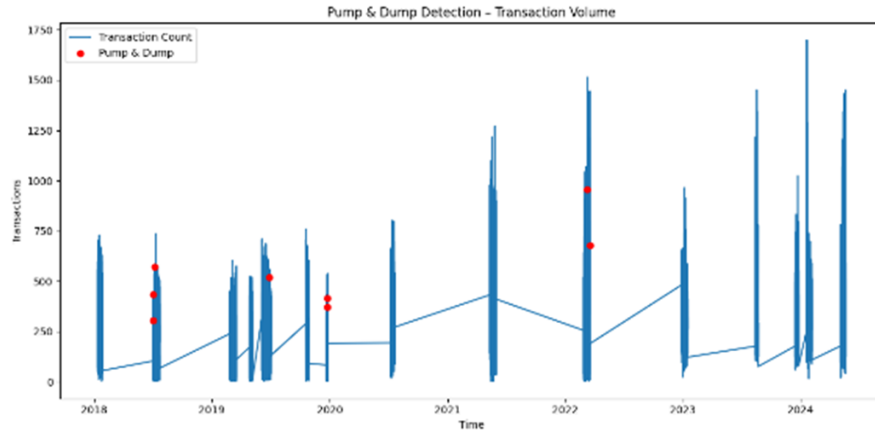


Figure 4.17: Pump-and-Dump transaction Volume

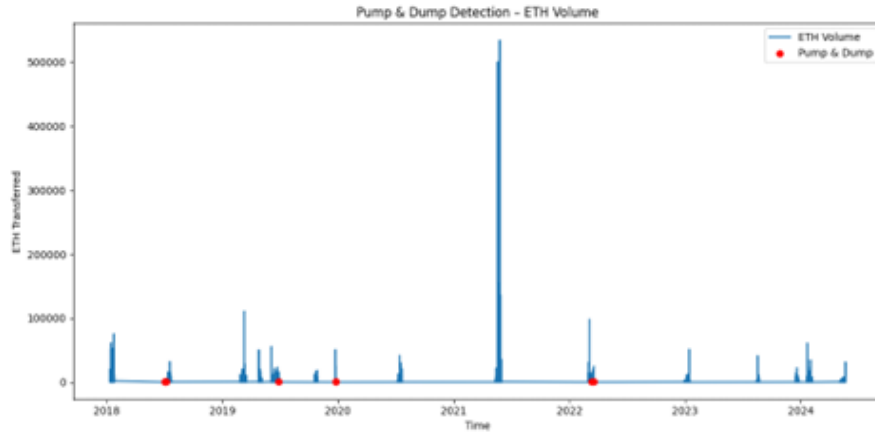


Figure 4.18: Pump-and-Dump ETH volume

4.14 Future Work

Future enhancements to this work can further strengthen the effectiveness and applicability of the proposed Ethereum blockchain anomaly detection framework. One promising direction is the integration of temporal Graph Neural Networks (GNNs) to better capture evolving transaction patterns and dynamic wallet interactions over time.

Incorporating real-time streaming analytics using frameworks such as Spark Structured Streaming would enable early detection of ongoing fraudulent activities instead of relying solely on batch analysis. Extending the system to support multi-chain data, including cross-chain bridges and decentralized finance (DeFi) protocols, can improve the detection of complex laundering activities that span multiple blockchains.

Additionally, adaptive thresholding mechanisms and reinforcement learning techniques can be explored to reduce false positives and dynamically adjust detection sensitivity. Finally, collaboration with on-chain labeling services and regulatory datasets can help validate detected anomalies, enhance model evaluation, and improve overall detection accuracy.

4.15 Conclusion

This study presented a comprehensive and scalable framework for detecting suspicious and anomalous activities in Ethereum blockchain transaction data by combining Big Data analytics, machine learning, deep learning, and graph-based techniques. The system successfully identified multiple forms of fraudulent and abnormal behavior, including wash trading, circular transactions, smurfing, anomalous wallet behavior, and pump-and-dump schemes.

Distributed data processing using PySpark enabled efficient feature extraction and aggregation over large-scale blockchain datasets. Graph-based network analysis revealed complex transactional relationships and coordinated behaviors that are not easily detectable through traditional statistical methods. Furthermore, the autoencoder-based anomaly detection model effectively identified wallets with abnormal transaction patterns without relying on labeled training data.

Overall, the experimental results demonstrate that integrating statistical analysis, graph theory, and deep learning provides a reliable, scalable, and interpretable approach for blockchain forensics and financial crime detection within decentralized networks such as Ethereum.

References

1. Y. Wu et al., “Behavior-aware Account De-anonymization on Ethereum Interaction Graph,” *arXiv preprint arXiv:2203.09360*, 2022.
2. S. Patel et al., “Blockchain-based Fraud Detection: A Systematic Review of Ethereum Network Applications,” *Cluster Computing*, Springer, 2025.
3. Y. Zhang et al., “Aparecium: Detecting Scam Behaviors on Ethereum via Biased Random Walk,” *Cybersecurity*, SpringerOpen, vol. 6, no. 1, 2023.
4. R. Kumar et al., “Detection of Fraudulent Transactions in Ethereum Blockchain Smart Contracts Using Deep Learning,” *Journal of King Saud University – Computer and Information Sciences*, 2025.
5. K. Reddy et al., “Intelligent Fraud Detection in Ethereum Transactions Using Machine Learning,” *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 2025.

List of Publications Based on This Research Work

1. Y. Wu et al., “Behavior-aware Account De-anonymization on Ethereum Interaction Graph,” *arXiv preprint arXiv:2203.09360*, 2022.
2. S. Patel et al., “Blockchain-based Fraud Detection: A Systematic Review of Ethereum Network Applications,” *Cluster Computing*, Springer, 2025.
3. Y. Zhang et al., “Aparecium: Detecting Scam Behaviors on Ethereum via Biased Random Walk,” *Cybersecurity*, SpringerOpen, vol. 6, no. 1, 2023.
4. R. Kumar et al., “Detection of Fraudulent Transactions in Ethereum Blockchain Smart Contracts Using Deep Learning,” *Journal of King Saud University – Computer and Information Sciences*, 2025.
5. K. Reddy et al., “Intelligent Fraud Detection in Ethereum Transactions Using Machine Learning,” *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 2025.

Chapter 5

Appendix

13. APPENDIX

Installations:

```
!apt-get install openjdk-11-jdk-headless -qq
!pip install pyspark
!pip install findspark
!pip install -q huggingface_hub fsspec
!pip install -q pyspark huggingface_hub
```

Version Check:

```
import pyspark
print("PySpark version:", pyspark.__version__)
```

Spark Session Creation:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("Ethereum Big Data Anomaly Detection") \
    .config("spark.executor.memory", "4g") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()
```

```
print("Spark Session Created Successfully")
```

Data Loading:

```
from huggingface_hub import list_repo_files, hf_hub_download
import os
```

```
repo_id = "vnegi10/Ethereum_blockchain_parquet"
base_dir = "/content/eth_full_data"
```



```

os.makedirs(base_dir, exist_ok=True)

files = list_repo_files(repo_id=repo_id, repo_type="dataset")

print("Total files in dataset:", len(files))

for file in files:
    if file.startswith("blocks/") or file.startswith("transactions/"):
        hf_hub_download(
            repo_id=repo_id,
            filename=file,
            repo_type="dataset",
            local_dir=base_dir
        )

```

Data Ingestion:

```

blocks_df = spark.read.parquet(
    "/content/eth_full_data/blocks/*.parquet"
)

tx_df = spark.read.parquet(
    "/content/eth_full_data/transactions/*.parquet"
)

```

Data Storage:

```

blocks_df = spark.read \
    .option("ignoreCorruptFiles", "true") \
    .parquet("/content/eth_full_data/blocks/*.parquet")

tx_df = spark.read \
    .option("ignoreCorruptFiles", "true") \
    .parquet("/content/eth_full_data/transactions/*.parquet")

blocks_df.describe().show()
tx_df.describe().show()

```

HDFS Layer:

```

blocks_df.write.mode("overwrite").parquet("/content/hdfs/blocks")
tx_df.write.mode("overwrite").parquet("/content/hdfs/transactions")

print("Data stored in Hadoop-style Parquet format")

```

Data Integration:

```
from pyspark.sql.functions import col
```

```
blocks_sel = blocks_df.select(
    "block_number",
    "timestamp",
    col("gas_used").alias("block_gas_used"),
    "base_fee_per_gas"
)
```

```
tx_sel = tx_df.select(
    "block_number",
    "from_address",
    "to_address",
    col("value_f64").alias("tx_value"),
    "gas_limit",
    "gas_used",
    "gas_price",
    "success"
)
```

```
eth_df = tx_sel.join(blocks_sel, "block_number", "inner")
```

```
eth_df.show(5)
eth_df.printSchema()
```

Data Preprocessing:

```
from pyspark.sql.functions import col, count, when
```

```
def null_report(df):
    df.select([
        count(when(col(c).isNull(), c)).alias(c)
        for c in df.columns
    ]).show(truncate=False)
```

```
print("Blocks null report")
null_report(blocks_df)
```

```
print("Transactions null report")
null_report(tx_df)
```

```
tx_clean = tx_df.dropna(
```

```

subset=[
    "value_f64",
    "gas_limit",
    "gas_used",
    "gas_price",
    "from_address",
    "to_address"
]
)

print("Before cleaning:", tx_df.count())
print("After cleaning:", tx_clean.count())

eth_clean = eth_df.fillna(0)

eth_clean = eth_clean.withColumn(
    "is_failed",
    (~col("success")).cast("int")
)

eth_feat = eth_clean.withColumn(
    "eth_value", col("tx_value") / 1e18
).withColumn(
    "tx_fee", col("gas_used") * col("gas_price")
).withColumn(
    "gas_utilization", col("gas_used") / col("block_gas_used")
)

eth_feat.select(
    "eth_value", "tx_fee", "gas_utilization"
).show(5, truncate=False)

eth_feat.select(
    "eth_value", "tx_fee", "gas_utilization"
).describe().show()

eth_feat.select("eth_value") \
    .sample(0.01) \
    .toPandas() \
    .hist(bins=50)

```

Transaction level Anomalies (whale transactions):

```

eth_feat.filter(col("eth_value") > 1000).show(10, truncate=False)

import matplotlib.pyplot as plt

pdf_high = eth_feat.filter(col("eth_value") > 1000) \
    .select(col("eth_value").cast("double")) \
    .sample(0.2) \
    .toPandas()

plt.figure(figsize=(8,5))
plt.hist(pdf_high["eth_value"], bins=30)
plt.title("High-Value Ethereum Transactions (>1000 ETH)")
plt.xlabel("ETH Value")
plt.ylabel("Frequency")
plt.show()

from pyspark.sql.functions import when
eth_feat = eth_feat.withColumn(
    "is_failed",
    when(col("success") == False, 1).otherwise(0)
)
eth_feat.groupBy("from_address") \
    .sum("is_failed") \
    .withColumnRenamed("sum(is_failed)", "failed_tx_count") \
    .orderBy(col("failed_tx_count").desc()) \
    .show(10, truncate=False)
import matplotlib.pyplot as plt
# Convert Spark → Pandas safely
pdf_failed = eth_feat.groupBy("from_address") \
    .sum("is_failed") \
    .withColumnRenamed("sum(is_failed)", "failed_tx_count") \
    .orderBy(col("failed_tx_count").desc()) \
    .limit(10) \
    .toPandas()
# Convert bytearray addresses to readable hex strings
pdf_failed["from_address"] = pdf_failed["from_address"].apply(
    lambda x: x.hex() if isinstance(x, (bytes, bytearray)) else str(x)
)
# Create numeric x positions
x_pos = range(len(pdf_failed))
plt.figure(figsize=(10,5))
plt.bar(x_pos, pdf_failed["failed_tx_count"])

```

```

plt.xticks(x_pos, pdf_failed["from_address"], rotation=45, ha="right")
plt.title("Top 10 Wallets with Failed Transactions")
plt.ylabel("Failed Transaction Count")
plt.xlabel("Wallet Address")
plt.tight_layout()
plt.show()
Address level behaviour Profiles:
from pyspark.sql.functions import avg, count, variance, col
address_features = eth_feat.groupBy("from_address").agg(
    count("*").alias("tx_count"),
    avg(col("eth_value").cast("double")).alias("avg_value"),
    avg(col("is_failed").cast("double")).alias("failure_rate"),
    variance(col("gas_used").cast("double")).alias("gas_variance")
)
address_features.show(5, truncate=False)
import matplotlib.pyplot as plt
pdf_addr = address_features.orderBy(col("tx_count").desc()) \
    .limit(10) \
    .toPandas()
# Convert bytearray addresses
pdf_addr["from_address"] = pdf_addr["from_address"].apply(
    lambda x: x.hex() if isinstance(x, (bytes, bytearray)) else str(x)
)
plt.figure(figsize=(10,5))
plt.bar(range(len(pdf_addr)), pdf_addr["tx_count"])
plt.xticks(range(len(pdf_addr)), pdf_addr["from_address"], rotation=45, ha="right")
plt.title("Top 10 Wallets by Transaction Count")
plt.ylabel("Transaction Count")
plt.xlabel("Wallet Address")
plt.tight_layout()
plt.show()
from pyspark.sql.functions import when
bot_wallets = address_features.withColumn(
    "is_bot",
    when(
        (col("tx_count") > 1000) &
        (col("avg_value") < 0.01),
        1
    ).otherwise(0)
)
bot_wallets.select(
    "from_address", "tx_count", "avg_value", "failure_rate", "is_bot"

```

```

).show(10, truncate=False)
detected_bots = bot_wallets.filter(col("is_bot") == 1)
print("Detected bot wallets count:", detected_bots.count())
detected_bots.show(10, truncate=False)
import matplotlib.pyplot as plt
pdf_bots = detected_bots.orderBy(col("tx_count").desc()) \
    .limit(10) \
    .toPandas()
pdf_bots["from_address"] = pdf_bots["from_address"].apply(
    lambda x: x.hex() if isinstance(x, (bytes, bytearray)) else str(x)
)
plt.figure(figsize=(10,5))
plt.bar(range(len(pdf_bots)), pdf_bots["tx_count"])
plt.xticks(range(len(pdf_bots)), pdf_bots["from_address"], rotation=45, ha="right")
plt.title("Top 10 Bot Wallets by Transaction Count")
plt.ylabel("Transaction Count")
plt.xlabel("Wallet Address")
plt.tight_layout()
plt.show()
Block level Anomaly:
from pyspark.sql.functions import max
block_anomaly = eth_feat.groupBy("block_number").agg(
    avg("gas_used").alias("avg_gas_used"),
    max("block_gas_used").alias("block_gas_used"),
    avg("base_fee_per_gas").alias("avg_base_fee"),
    count("*").alias("tx_count")
).withColumn(
    "block_gas_utilization",
    col("avg_gas_used") / col("block_gas_used")
)
block_anomaly.show(5, truncate=False)
import matplotlib.pyplot as plt
pdf_block = block_anomaly.sample(0.01).toPandas()
plt.figure(figsize=(8,5))
plt.scatter(pdf_block["tx_count"], pdf_block["avg_gas_used"])
plt.xlabel("Transaction Count per Block")
plt.ylabel("Average Gas Used")
plt.title("Block Activity vs Gas Usage")
plt.tight_layout()
plt.show()
Isolation Forest Code:
from pyspark.sql.functions import avg, count, variance, col

```

```

# Aggregate features per wallet
wallet_features = eth_feat.groupBy("from_address").agg(
    count("*").alias("tx_count"),
    avg(col("eth_value").cast("double")).alias("avg_value"),
    avg(col("is_failed").cast("double")).alias("failure_rate"),
    variance(col("gas_used").cast("double")).alias("gas_variance")
).fillna(0)
wallet_features.show(5, truncate=False)
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
# Collect wallet features to Pandas
pdf_wallet_features = address_features.select(
    "tx_count", "avg_value", "failure_rate", "gas_variance", "from_address"
).toPandas()
# Run sklearn Isolation Forest
clf = IsolationForest(contamination=0.02, random_state=42)
clf.fit(pdf_wallet_features[["tx_count", "avg_value", "failure_rate", "gas_variance"]])
pdf_wallet_features["anomaly"] = clf.predict(
    pdf_wallet_features[["tx_count", "avg_value", "failure_rate", "gas_variance"]]
)
# Convert sklearn labels: -1 = anomaly, 1 = normal
pdf_wallet_features["is_anomaly"] = (pdf_wallet_features["anomaly"] == -1).astype(int)
# Top 10 suspicious wallets
suspicious_wallets = pdf_wallet_features[pdf_wallet_features["is_anomaly"] == 1] \
    .sort_values("tx_count", ascending=False).head(10)
plt.figure(figsize=(10,5))
plt.bar(range(len(suspicious_wallets)), suspicious_wallets["tx_count"])
plt.xticks(range(len(suspicious_wallets)), suspicious_wallets["from_address"], rotation=45)
plt.title("Top 10 Suspicious Wallets (Isolation Forest)")
plt.ylabel("Transaction Count")
plt.xlabel("Wallet Address")
plt.tight_layout()
plt.show()
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
from pyspark.sql.functions import col
# 1 Collect features from Spark to Pandas
pdf_wallet_features = address_features.select(
    "from_address", "tx_count", "avg_value", "failure_rate", "gas_variance"
).toPandas()
# 2 Fit Isolation Forest
clf = IsolationForest(contamination=0.02, random_state=42)

```

```

clf.fit(pdf_wallet_features[["tx_count", "avg_value", "failure_rate", "gas_variance"]])

# 3 Predict anomalies
pdf_wallet_features["anomaly"] = clf.predict(
    pdf_wallet_features[["tx_count", "avg_value", "failure_rate", "gas_variance"]]
)
# 4 Convert labels: -1 = anomaly, 1 = normal
pdf_wallet_features["is_anomaly"] = (pdf_wallet_features["anomaly"] == -1).astype(int)
# 5 Filter suspicious wallets
wallet_anomaly = pdf_wallet_features
# 6 Scatter plot: Transaction count vs ETH value
plt.figure(figsize=(10,6))
plt.scatter(
    wallet_anomaly["tx_count"],
    wallet_anomaly["avg_value"],
    c=wallet_anomaly["is_anomaly"],
    cmap="coolwarm",
    alpha=0.6
)
plt.xlabel("Transaction Count")
plt.ylabel("Average ETH Value")
plt.title("Wallet Anomalies (Red = Suspicious)")
plt.colorbar(label="Anomaly (1 = Suspicious)")
plt.show()
Autoencoder Code:
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import MLPRegressor
import matplotlib.pyplot as plt
# 1. Collect wallet features into Pandas
pdf_wallet = wallet_features.select(
    "from_address", "tx_count", "avg_value", "failure_rate", "gas_variance"
).toPandas()
# Convert addresses to strings
pdf_wallet["from_address"] = pdf_wallet["from_address"].apply(
    lambda x: x.hex() if isinstance(x, (bytes, bytearray)) else str(x)
)
# 2. Normalize features
features = ["tx_count", "avg_value", "failure_rate", "gas_variance"]
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(pdf_wallet[features])

```



```

# 3. Build Autoencoder (MLPRegressor)
autoencoder = MLPRegressor(hidden_layer_sizes=(8,4,8), max_iter=500, random_state=42)
autoencoder.fit(X_scaled, X_scaled) # Train to reconstruct input
# 4. Compute reconstruction error
X_pred = autoencoder.predict(X_scaled)
recon_error = np.mean(np.square(X_scaled - X_pred), axis=1)
pdf_wallet["recon_error"] = recon_error
# 5. Mark anomalies (top 2% error)
threshold = np.percentile(recon_error, 98)
pdf_wallet["anomaly"] = (pdf_wallet["recon_error"] >= threshold).astype(int)
# 6. Plot top anomalous wallets
anomalous_wallets = pdf_wallet[pdf_wallet["anomaly"]==1].sort_values("tx_count", ascending=False)
plt.figure(figsize=(12,6))
plt.bar(anomalous_wallets["from_address"], anomalous_wallets["tx_count"], color='red')
plt.xticks(rotation=90)
plt.xlabel("Wallet Address")
plt.ylabel("Transaction Count")
plt.title("Top Anomalous Wallets (Autoencoder)")
plt.tight_layout()
plt.show()
Money Laundering Patterns:
from pyspark.sql.functions import col, count, avg
import networkx as nx
import matplotlib.pyplot as plt
# -----
# Prepare edges for transactions
# -----
edges = eth_feat.select(
    col("from_address").alias("src"),
    col("to_address").alias("dst"),
    col("eth_value"),
    col("block_number")
).filter(col("eth_value") > 0)
# -----
# Detect Smurfing Wallets
# -----
smurfing = edges.groupBy("src").agg(
    count("*").alias("tx_count"),
    avg("eth_value").alias("avg_tx_value")
).filter((col("tx_count") > 50) & (col("avg_tx_value") < 0.1)) # threshold adjustable
print("Top 10 Smurfing Wallets:")
smurfing.show(10, truncate=False)

```

```

# Collect smurfing wallet addresses for highlighting
smurf_nodes = smurfing.select("src").rdd.flatMap(
    lambda x: [x[0].hex() if isinstance(x[0], bytearray) else str(x[0])]
).collect()
# -----
# Build Transaction Graph (Sample for Visualization)
# -----
# Build Transaction Graph (Sample for Visualization)
# -----
pdf_edges = edges.limit(5000).toPandas() # limit for graph processing
G = nx.DiGraph()
for _, row in pdf_edges.iterrows():
    src = row["src"].hex() if isinstance(row["src"], bytearray) else str(row["src"])
    dst = row["dst"].hex() if isinstance(row["dst"], bytearray) else str(row["dst"])
    G.add_edge(src, dst, weight=row["eth_value"])
# -----
# Detect Circular Transfers (Cycles)
# -----
cycles = list(nx.simple_cycles(G))
print(f"\nNumber of circular transfer cycles detected: {len(cycles)}\n")
# Prepare edges involved in cycles
cycle_edges = []
for cycle in cycles:
    for i in range(len(cycle)):
        cycle_edges.append((cycle[i], cycle[(i+1) % len(cycle)]))
# -----
# Filter nodes/edges for visualization
# -----
# Only include smurfing wallets present in the sampled graph
smurf_nodes_in_graph = [n for n in smurf_nodes if n in G.nodes()]
# Only include cycle edges that exist in the sampled graph
cycle_edges_in_graph = [(u, v) for u, v in cycle_edges if u in G.nodes() and v in G.nodes()]
# -----
# Visualize Transaction Graph
# -----
pos = nx.spring_layout(G, seed=42) # consistent layout

plt.figure(figsize=(12,10))
# Draw all nodes and edges lightly
nx.draw_networkx_nodes(G, pos, node_size=50, node_color='lightblue', alpha=0.7)
nx.draw_networkx_edges(G, pos, alpha=0.1)

```

```

# Highlight smurfing wallets
if smurf_nodes_in_graph:
    nx.draw_networkx_nodes(
        G, pos,
        nodelist=smurf_nodes_in_graph,
        node_size=80,
        node_color='red',
        label='Smurfing Wallets'
    )
# Highlight circular transfers
if cycle_edges_in_graph:
    nx.draw_networkx_edges(
        G, pos,
        edgelist=cycle_edges_in_graph,
        edge_color='orange',
        width=2,
        label='Circular Transfers'
    )
plt.title("Ethereum Transaction Graph (Sample, Smurfing & Circular Transfers Highlight
plt.legend()
plt.show()
Pump and Dump Detection:
from pyspark.sql.functions import col, from_unixtime
eth_time = eth_feat.withColumn(
    "event_time",
    from_unixtime(col("timestamp")).cast("timestamp")
)
eth_time.select("event_time", "eth_value").show(5, truncate=False)
from pyspark.sql.functions import window, count, sum, avg
time_agg = eth_time.groupBy(
    window(col("event_time"), "10 minutes")
).agg(
    count("*").alias("tx_count"),
    sum("eth_value").alias("total_eth"),
    avg("eth_value").alias("avg_eth")
).orderBy("window")
time_agg.show(5, truncate=False)
pdf = time_agg.select(
    col("window.start").alias("time"),
    "tx_count",
    "total_eth",
    "avg_eth"

```

```

).toPandas()
pdf = pdf.sort_values("time")
pdf.head()
# 1 hour baseline = 6 × 10-minute windows
pdf["rolling_tx"] = pdf["tx_count"].rolling(6).mean()
pdf["rolling_eth"] = pdf["total_eth"].rolling(6).mean()
# Pump indicators
pdf["tx_spike"] = (pdf["tx_count"] - pdf["rolling_tx"]) / pdf["rolling_tx"]
pdf["eth_spike"] = (pdf["total_eth"] - pdf["rolling_eth"]) / pdf["rolling_eth"]
# Dump indicator (next window crash)
pdf["future_tx_change"] = pdf["tx_spike"].shift(-1)
pdf["pump_dump"] = (
    (pdf["tx_spike"] > 1.5) &          # sudden pump
    (pdf["eth_spike"] > 1.5) &        # high ETH inflow
    (pdf["future_tx_change"] < -0.5) # sharp drop after
).astype(int)
pump_events = pdf[pdf["pump_dump"] == 1]
print(" Pump & Dump Events Detected:", len(pump_events))
pump_events[["time", "tx_count", "total_eth", "tx_spike", "eth_spike"]].head()
import matplotlib.pyplot as plt
plt.figure(figsize=(12,6))
plt.plot(pdf["time"], pdf["tx_count"], label="Transaction Count")
plt.scatter(
    pump_events["time"],
    pump_events["tx_count"],
    color="red",
    label="Pump & Dump",
    zorder=3
)
plt.xlabel("Time")
plt.ylabel("Transactions")
plt.title("Pump & Dump Detection { Transaction Volume}")
plt.legend()
plt.tight_layout()
plt.show()
plt.figure(figsize=(12,6))
plt.plot(pdf["time"], pdf["total_eth"], label="ETH Volume")
plt.scatter(
    pump_events["time"],
    pump_events["total_eth"],
    color="red",
    label="Pump & Dump",

```

```
        zorder=3
    )
    plt.xlabel("Time")
    plt.ylabel("ETH Transferred")
    plt.title("Pump & Dump Detection { ETH Volume}")
    plt.legend()
    plt.tight_layout()
    plt.show()
```